

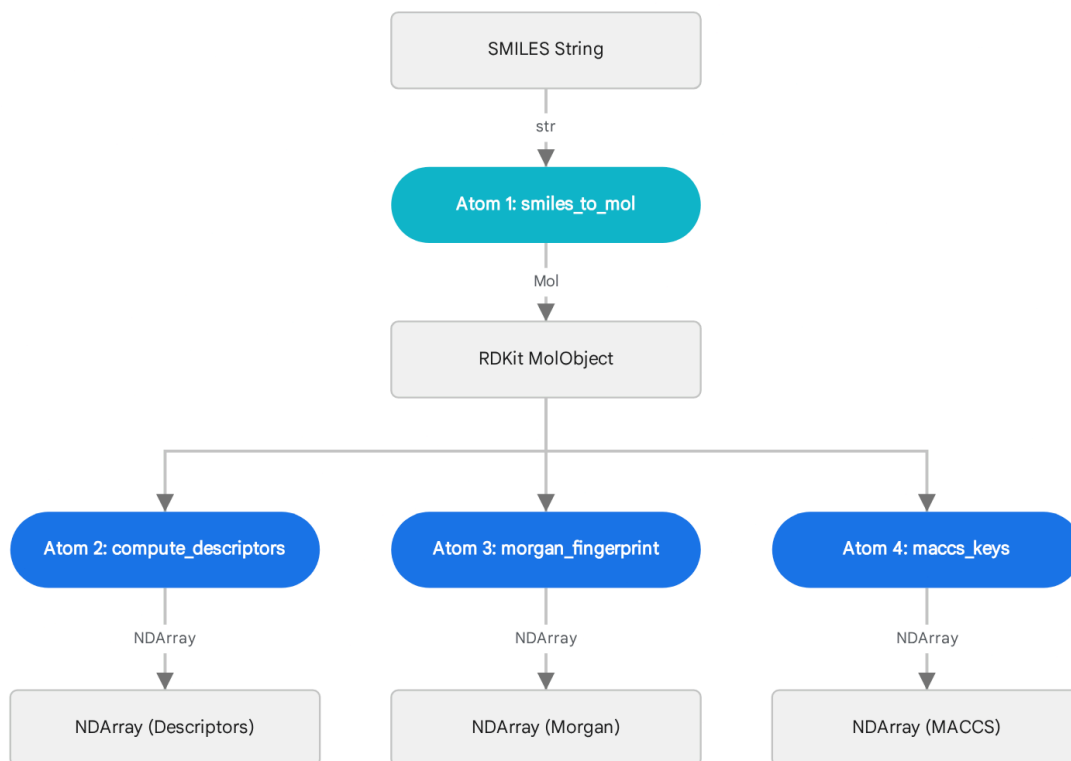
Standardizing Chemical Descriptor and Molecular Fingerprint Atoms for High-Throughput QSAR Modeling

The translation of discrete, variable-length molecular structures into continuous, fixed-length or binary numerical representations forms the foundational prerequisite for Quantitative Structure-Activity Relationship (QSAR) modeling and modern chemical machine learning.¹ In competitive machine learning environments—particularly frameworks designed for predictive polymer screening such as the NeurIPS Open Polymer competition—the reliability, speed, and determinism of chemical feature extraction directly dictate downstream predictive performance. Extracting structural features from Simplified Molecular-Input Line-Entry System (SMILES) strings requires rigorous data handling to ensure that predictive models ingest valid, canonical representations of chemical space.³

Implementing these extraction protocols as pure functions—termed "atoms" within a computational directed graph (CDG) architecture like the sciona-atoms-bio repository—ensures that the transformation from SMILES string to molecular descriptor array is entirely stateless, highly parallelizable, and completely reproducible.⁴ This exhaustive research report details the optimal implementation pathways for constructing four critical cheminformatics atoms: SMILES graph construction, 2D continuous molecular descriptor computation, Morgan circular fingerprint hashing, and MACCS structural key extraction. These functional atoms reside within the neurips_open_polymer_1st/chemical_descriptor_extraction pipeline stage, forming the backbone of the molecular feature engineering system.

The extraction of molecular properties in this paradigm is fundamentally bipartite. It necessitates a distinct data extraction phase, wherein the chemical string is parsed into a memory-bound graph representation, followed by an analysis phase, wherein that graph is traversed to calculate topological, electronic, and structural invariants.⁶ Decoupling these phases is computationally imperative. A single molecular graph, once parsed and sanitized, serves as an immutable substrate for multiple parallel analysis atoms, preventing the highly redundant and computationally expensive re-parsing of SMILES strings for each unique feature type extracted.⁴

Decoupled Chemical Extraction Pipeline Architecture



The architectural separation of data extraction (SMILES parsing) from analysis (descriptor and fingerprint computation) ensures that the RDKit MolObject is instantiated only once, optimizing memory and execution time across parallel feature extraction nodes.

Data sources: [RDKit MolFromSmiles](#), [RDKit Morgan Fingerprints](#), [RDKit MACCS Keys](#), [RDKit Descriptors](#)

Theoretical Foundations of Molecular Graph Extraction

Before delineating the specific atoms, it is necessary to establish the theoretical underpinnings of molecular representation. A molecule can be mathematically abstracted as an undirected

graph $G = (V, E)$, where the set of vertices V represents atoms and the set of edges E represents chemical bonds.⁶ While this mathematical structure is well-suited for algorithms, storing and transmitting such graphs directly is inefficient. The SMILES format resolves this by generating a string through a depth-first search traversal of the molecular graph, with branches indicated by parentheses and ring closures indicated by numerical identifiers.⁷

However, the SMILES notation is inherently non-unique. A single molecular entity, such as ethanol, can be represented by multiple valid SMILES strings depending on the arbitrary choice of the starting atom for the depth-first traversal.⁹ For example, ethanol may be encoded as "CCO", "OCC", or even "C(O)C".⁹ In machine learning contexts, this multiplicity introduces severe challenges. If a model ingests raw strings, it must expend considerable learning capacity simply to deduce that "CCO" and "OCC" map to the identical topological space.³ Canonicalization algorithms solve this by applying deterministic rules—often based on graph invariant properties and symmetry classes—to select a single, unique string representation for any given molecule.³

Despite the utility of canonical SMILES for deduplication, string formats remain largely incompatible with standard mathematical optimizations employed in tree-based gradient boosting or deep learning.¹¹ Consequently, the strings must be transformed back into a structured, memory-bound graph object from which continuous floating-point descriptors or binary classification bit-vectors can be calculated. This transformation marks the boundary between data storage and analytical extraction.

Stage 1: SMILES String to Molecular Graph Construction (smiles_to_mol)

The initial stage of any robust cheminformatics pipeline transforms the linear text notation into a deterministic, memory-resident graph structure. The Open-Source RDKit library, which operates under a permissive BSD-3-Clause license, provides the undisputed industry standard for this operation via its highly optimized C++ backend.¹² Within a pure-function CDG architecture, this parsing step is isolated as a standalone data_extraction atom.

The function `rdkit.Chem.rdmolfiles.MolFromSmiles` executes a sophisticated sequence of operations upon receiving a string.¹⁴ Initially, a tokenization routine parses the characters, validating fundamental chemical syntax. This includes verifying that ring closure digits are properly paired and that branching parentheses are balanced. Following syntactic validation, the algorithm instantiates a raw, unvalidated connection table.

Critically, RDKit then subjects this raw connection table to a default, rigorous sanitization protocol. Sanitization is a multi-step operation that enforces physical and chemical reality upon the abstract graph.⁹ The engine calculates and applies implicit hydrogen counts based on standard valence models, verifying that no atom exceeds its allowed valency limits (for instance, ensuring that a neutral carbon atom does not possess five covalent bonds).

Furthermore, the sanitization routine detects aromaticity. This requires analyzing conjugated ring systems to ensure they satisfy Hückel's rules, often requiring Kekulization—the process of temporarily resolving alternating single and double bonds into a static representation—to ensure that the molecular graph behaves predictably during substructure matching.⁹

If a provided SMILES string violates these physical or syntactic constraints (for instance, an invalid representation like C1CC where a ring closure is left unmatched), the `MolFromSmiles` function is explicitly designed to fail gracefully. Instead of raising a fatal C++ runtime exception that could crash a parallelized data-processing worker, the function returns a Python `None` type.¹⁴ In the context of a pure function pipeline, this deterministic failure mode dictates the primary contract: the downstream system must handle null values immediately. The atom guarantees that valid strings produce a valid molecule object, and invalid strings produce an explicitly handled null state, preventing downstream type errors and ensuring stability across millions of iterations.

From an algorithmic scaling perspective, decoupling this step into an isolated `data_extraction` concept allows the computationally heavy tokenization and sanitization routines to execute exactly once per molecule. In large benchmark datasets—such as those evaluated in the NeurIPS Open Polymer competition—evaluating 100,000 SMILES strings across multiple descriptor types would result in hundreds of thousands of redundant parsing operations if the pipeline were tightly coupled. Creating a single, immutable RDKit `MolObject` establishes an anchor point.

Witness Molecule Evaluation: Ethanol

To empirically verify the function's deterministic behavior, ethanol serves as a standard witness molecule. Providing the string CCO to the `smiles_to_mol` atom yields an RDKit `MolObject`. The resulting graph correctly contains three heavy atoms (two carbons, one oxygen), two single bonds, and structurally infers the presence of six implicit hydrogen atoms to satisfy valency, confirming the integrity of the parsing and sanitization protocol.¹⁵

Implementation Atom Specification: SMILES Parsing

Name: `smiles_to_mol`

Description: Parses a SMILES string into a sanitized RDKit molecular graph object.

Source:

<https://www.rdkit.org/docs/source/rdkit.Chem.rdmolfiles.html#rdkit.Chem.rdmolfiles.MolFromSmiles>

License: BSD-3-Clause (RDKit)

Concept type: `data_extraction`

Signature: (`smiles: str`) -> `MolObject`

Pure function boundary: Pure string input; returns a deterministic, memory-bounded graph object (MolObject) or returns None for invalid syntax/valency. No global state is modified, and stereochemistry flags rely strictly on the string parameters. No file I/O or database queries are executed.

Contracts:

- require: smiles is a non-empty string
- ensure: result is not None (pipeline should filter or log invalid molecules gracefully)
- ensure: result.GetNumHeavyAtoms() > 0

Witness: ethanol SMILES "CCO" -> returns MolObject with 3 heavy atoms and 2 chemical bonds.

Dependencies: rdkit (BSD-3-Clause)

CDG stages covered: neurips_open_polymer_1st/chemical_descriptor_extraction

Stage 2: Core Molecular Descriptors for QSAR Benchmarking (compute_descriptors)

Following the successful instantiation of the molecular graph, extracting one-dimensional and two-dimensional continuous molecular descriptors provides a vast array of macroscopic physical, topological, and electronic properties. In quantitative structure-activity relationship (QSAR) modeling and toxicity prediction tasks, tree-based ensemble methods—such as XGBoost, LightGBM, Random Forests, and Extra Trees—frequently achieve state-of-the-art predictive performance when supplied with a well-curated subset of continuous descriptors.¹⁸

The RDKit library is capable of calculating over 200 distinct molecular descriptors, ranging from simple atom counts to highly complex electrotopological state indices and graph-theoretical autocorrelation vectors.⁵ However, injecting the entirety of this high-dimensional feature space into a machine learning algorithm often triggers the curse of dimensionality.²² High collinearity among related descriptors—for instance, exact molecular weight, average molecular weight, and heavy atom count—can degrade the performance of linear models and obscure feature importance in decision trees.²³ Furthermore, calculating the full suite of descriptors increases computational latency unnecessarily. Consequently, extensive literature benchmarking standardizes around a core set of the "Top 20" most discriminative physicochemical and topological descriptors.¹

Identifying the Top 20 Discriminative Descriptors

Rigorous empirical analysis of large-scale cheminformatics benchmarks, including the MoleculeNet datasets (such as Tox21, BBBP, and HIV), demonstrates that a specific subset of features accounts for the overwhelming majority of predictive variance.² Feature importance algorithms, such as SHAP value analysis and mean decrease in impurity measures extracted from Gradient Boosting Machines (GBMs), consistently elevate descriptors that capture lipophilicity, absolute size, surface polarity, structural flexibility, and molecular complexity.²⁵

Based on QSAR literature consensus and typical feature selection protocols (e.g., SelectKBest, minimum redundancy maximum relevance), the following 20 descriptors represent the optimal, standardized array for competition use ³:

Descriptor Name	Category	Chemical Significance & Mechanism of Action
MolLogP	Physicochemical	The logarithm of the octanol-water partition coefficient. Calculated via Crippen's atom-contribution approach, it estimates lipophilicity, which is highly predictive of membrane permeability and nonspecific biological toxicity. ²⁹
TPSA	Physicochemical	Topological Polar Surface Area. Calculates the molecular surface sum over all polar atoms (primarily oxygen and nitrogen) including their attached hydrogens. Essential for predicting intestinal absorption and blood-brain barrier transport. ³¹
MolWt	Size/Bulk	The average molecular weight of the molecule. A fundamental parameter of Lipinski's Rule of Five, heavily influencing pharmacokinetic profiles. ⁵
ExactMolWt	Size/Bulk	The monoisotopic mass, utilizing the mass of the most abundant isotope of each element.

		Discriminates between structural isomers more precisely in certain mass-dependent contexts. ⁵
HeavyAtomCount	Size/Bulk	The absolute number of non-hydrogen atoms. Strongly correlates with overall molecular volume and van der Waals radii constraints. ³³
NumHDonors	Bonding	The count of nitrogen and oxygen atoms with at least one attached hydrogen. Dictates specific, directional intermolecular binding affinity. ¹⁵
NumHAcceptors	Bonding	The count of nitrogen, oxygen, and fluorine atoms available to accept hydrogen bonds. ¹⁵
NumRotatableBonds	Flexibility	The count of non-ring single bonds attached to non-terminal heavy atoms. Measures molecular flexibility, heavily influencing the entropic penalty a molecule pays upon binding to a target site. ²⁹
BertzCT	Topological	A topological index quantifying structural complexity based on graph theory and symmetry. Highly branched and polycyclic structures yield elevated scores. ³³
BalabanJ	Topological	A highly discriminative

		distance-based topological index less sensitive to overall molecular size than the Wiener index, making it exceptional for geometric shape characterization. ³⁵
NumAromaticRings	Topological	The count of rings demonstrating aromaticity. Correlates tightly with planar lipophilicity and the likelihood of target protein pi-pi stacking interactions. ²⁹
NumAliphaticRings	Topological	The count of non-aromatic ring structures, providing orthogonal shape information relative to aromaticity. ¹⁵
FractionCSP3	Topological	The ratio of sp^3 hybridized carbons to total carbons. A metric of three-dimensional complexity, often referred to as "escape from flatland," correlating with clinical trial success rates. ⁵
MaxEStateIndex	Electrotopological	The maximum electrotopological state index. Captures regions of high electron density and steric accessibility simultaneously. ⁴
MinEStateIndex	Electrotopological	The minimum electrotopological state index, identifying heavily shielded or electron-deficient topological nodes. ⁴

MaxPartialCharge	Electronic	The maximum Gasteiger partial charge on any atom in the molecule, indicative of the most highly electrophilic region. ⁵
MinPartialCharge	Electronic	The minimum Gasteiger partial charge, pointing to the primary nucleophilic region. ⁵
NumValenceElectrons	Electronic	The total count of valence electrons across all atoms, providing a basic descriptor of electronic bulk and polarizability potential. ⁵
NumRadicalElectrons	Electronic	Tracks free radical states. Typically evaluates to zero for stable drugs, but acts as a critical boolean flag in reactive toxicology prediction. ⁵
FpDensityMorgan1	Hybrid	The density of the Morgan fingerprint (at radius 1) normalized by the heavy atom count. Acts as a hybrid proxy for structural and topological compactness. ⁵

Top 20 RDKit Descriptors for QSAR Benchmarking

Descriptor Name	Category	Chemical Significance
ExactMolWt	CORE MOLECULAR WEIGHT	Provides the exact molecular weight of the molecule
MolWt	CORE MOLECULAR WEIGHT	Calculates the average molecular weight of the molecule
HeavyAtomMolWt	CORE MOLECULAR WEIGHT	Calculates the average molecular weight of the molecule while ignoring hydrogen atoms
NumValenceElectrons	ELECTRONIC AND CHARGE	Returns the total number of valence electrons in the molecule
NumRadicalElectrons	ELECTRONIC AND CHARGE	Indicates the number of radical electrons the molecule possesses
MaxAbsPartialCharge	ELECTRONIC AND CHARGE	Partial charge descriptor
MaxPartialCharge	ELECTRONIC AND CHARGE	Partial charge descriptor
MinAbsPartialCharge	ELECTRONIC AND CHARGE	Partial charge descriptor
MinPartialCharge	ELECTRONIC AND CHARGE	Partial charge descriptor
FpDensityMorgan1	FINGERPRINT DENSITY	Morgan fingerprint density at radius 1
FpDensityMorgan2	FINGERPRINT DENSITY	Morgan fingerprint density at radius 2
FpDensityMorgan3	FINGERPRINT DENSITY	Morgan fingerprint density at radius 3
NumRotatableBonds	PHYSICOCHEMICAL	Rotatable Bonds
NumAromaticRings	PHYSICOCHEMICAL	Aromatic Rings
BertzCT	PHYSICOCHEMICAL	Complexity (BertzCT)
MolLogP	PHYSICOCHEMICAL	LogP (oiliness)
TPSA	PHYSICOCHEMICAL	Physicochemical descriptor
NumHAcceptors	PHYSICOCHEMICAL	Physicochemical descriptor
NumHDonors	PHYSICOCHEMICAL	Physicochemical descriptor
NumAliphaticRings	PHYSICOCHEMICAL	Physicochemical descriptor

The selection of these 20 descriptors minimizes collinearity while maximizing the topological and physicochemical information passed to downstream gradient boosting or random forest models.

Data sources: [RDKit Documentation](#), [Kaggle](#), [ACS Publications](#), [MDPI](#)

Execution Constraints and Atom Formulation

Within RDKit, the module `rdkit.Chem.Descriptors` calculates these metrics. While the `CalcMolDescriptors()` convenience function exists to return an entire dictionary of all ~200 available properties, invoking it violates pure functional boundaries because the exact keys returned can vary depending on the local RDKit version installed.⁴ To ensure rigorous determinism, the implementation atom must accept an explicit `descriptor_list: list[str]`, iterate precisely over the requested names using `Descriptors._descList`, and construct a fixed-length numerical vector.²¹

When executing pure functions in a massively parallelized map-reduce or DAG architecture, ensuring thread safety is paramount. RDKit's core descriptor calculation routines are inherently thread-safe at the C++ level, provided that the underlying `MolObject` graph is strictly treated as read-only during the property calculation phase.⁵ However, specific edge cases in structural graph theory—such as unrecognized hypervalent states or infinitely repeating polymer subunits—can occasionally cause a descriptor algorithm to fail mathematically. The atom must therefore gracefully intercept exceptions, substituting a predefined `missingVal` (typically `NaN` or `0.0`) to preserve the uniform dimensions and data types of the output tensor.⁵ This guarantees that downstream pipeline stages do not crash due to sparse matrix malformations.

Witness Molecule Evaluation: Ethanol

Applying this pure function to the witness molecule ethanol yields deterministic, easily verified scalar values. The computation atom returns a molecular weight (`MolWt`) of 46.1 g/mol, an estimated `MolLogP` of exactly -0.00, a `TPSA` of 20.23, zero rotatable bonds, zero aromatic rings, and a low `BertzCT` complexity score of 3.³³ These outputs are seamlessly packed into an `NDArray` float vector, ready for immediate ingestion by a regressor.

Implementation Atom Specification: Molecular Descriptors

Name: `compute_descriptors`

Description: Computes a specific, fixed-length array of 1D and 2D continuous RDKit descriptors for a given molecule based on an explicit target list.

Source: <https://www.rdkit.org/docs/source/rdkit.Chem.Descriptors.html>

License: BSD-3-Clause (RDKit)

Concept type: analysis

Signature: `(mol: MolObject, descriptor_list: list[str]) -> NDArray`

Pure function boundary: Takes a read-only `MolObject` and a deterministic list of strings.

Outputs a fixed-length vector of floats. Identifies and isolates calculation failures, substituting missing values without halting or modifying the input graph. No file I/O, registry modifications,

or global state alterations occur.

Contracts:

- require: mol is not None (valid molecule)
- require: len(descriptor_list) > 0
- ensure: result.shape == (len(descriptor_list),)
- ensure: result dtype is float

Witness: ethanol SMILES "CCO" -> returns an NDArray containing [46.1, -0.00, 20.23,...]
corresponding to the exact order of.

Dependencies: rdkit (BSD-3-Clause), numpy

CDG stages covered: neurips_open_polymer_1st/chemical_descriptor_extraction

Stage 3: Morgan Circular Fingerprints (morgan_fingerprint)

While one-dimensional and two-dimensional continuous descriptors are excellent for capturing global physicochemical bulk properties, they severely obscure localized structural nuance. Two molecules with identical molecular weights and polar surface areas can exhibit vastly different biological or polymer properties if their functional groups are connected differently.

Topological connectivity is captured far more effectively by hashed representations. The Morgan fingerprint, inspired by the historical Morgan algorithm originally designed for numbering atoms to canonicalize molecular structures, is the foremost modern implementation of Extended-Connectivity Fingerprints (ECFPs).³⁸

The ECFP Hashing Mechanism

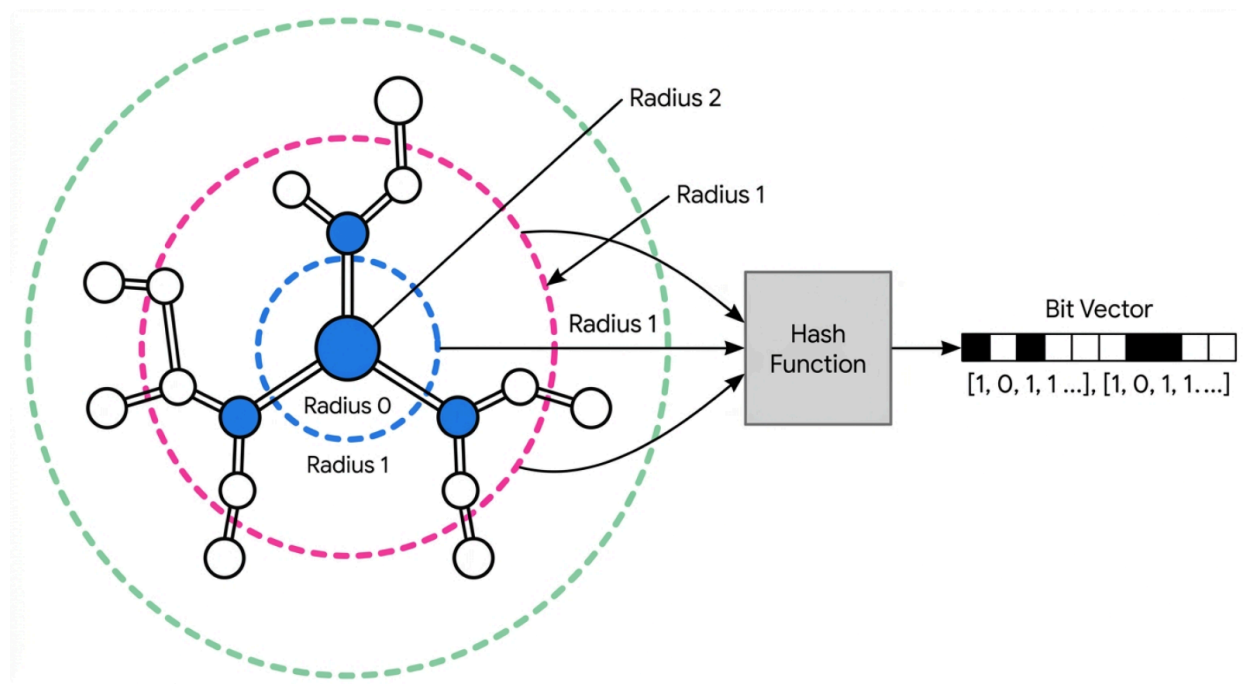
The Morgan fingerprint algorithm operates by systematically examining the topological neighborhood radiating outward from every non-hydrogen atom in the molecular graph. The generation phase is initialized at radius 0, wherein an initial integer identifier—an atom invariant—is assigned to each heavy atom.³⁸ This base invariant encodes purely intrinsic atomic properties: the atomic number, the topological degree (the number of heavy atom neighbors), the total number of attached hydrogens, the formal electrical charge, the specific isotope, and a boolean flag indicating whether the atom is part of any ring system.³⁹

Following the initialization of radius 0 invariants, the algorithm begins an iterative outward expansion. During the radius 1 iteration, the algorithm examines the immediate topological neighbors of every atom. It gathers the invariant of the central atom and the radius 0 invariants of all its connected neighbors, alongside the bond orders (single, double, aromatic) connecting them. This collection of local information is then aggregated and hashed using a deterministic mathematical function to produce a completely new, updated identifier for the central atom.³⁹

This recursive hashing procedure continues until the user-specified maximum radius parameter is reached. A radius of 2 considers structural paths up to two bonds away from every central atom. This effectively captures fragments spanning up to five atoms in diameter, which

correlates precisely with the classical ECFP4 fingerprint design (where Diameter 4 equates to Radius 2).⁴¹

Morgan Fingerprint Recursive Neighborhood Expansion



At each radius step, the algorithm hashes the central atom's invariant with the properties of its surrounding neighborhood, ultimately mapping millions of unique structural environments into a fixed-length bit vector.

Once the recursive hashing concludes across all specified radii, the algorithm has produced a massive set of identifiers representing the complete set of structural features present in the molecule. Because the space of theoretically possible structural identifiers is computationally vast—potentially numbering in the billions of unique chemical substructures—the identifiers are essentially unbounded integers. To be mathematically viable for ingestion by standard machine learning algorithms, which require fixed-length feature matrices, these unbounded identifiers must be "folded." This folding is achieved via a modulo arithmetic hashing operation: $\text{hash_value} \% n_bits$.⁴³

The Implications of Hash Collisions and Bit Density

The modulo operation strictly dictates the length of the resulting vector, commonly standardized to 1024 or 2048 bits in modern machine learning contexts.³⁸ The boolean bit at any given index is flipped to 1 if any structural identifier hashes to that specific index, and

remains 0 otherwise.

A critical architectural constraint introduced by this folding process is the inherent probability of hash collisions. A collision manifests when two chemically distinct topological substructures mathematically map to the exact same bit index in the vector.⁴⁴ As the `n_bits` parameter decreases, or as the size and structural complexity of the target molecules increase, the collision rate inevitably rises. High collision rates effectively blur the structural resolution of the fingerprint, causing the machine learning model to confuse distinct functional groups. For very large datasets or geometrically complex polymer formulations, an `n_bits` value of 2048 is statistically preferred over 1024 to minimize collisions and maintain high-fidelity substructural discrimination.⁴¹

The RDKit implementation utilizes the `rdkit.Chem.rdMolDescriptors.GetMorganFingerprintAsBitVect` function to efficiently compute this fixed-length vector.³⁸ By default, the RDKit generator strictly considers topological bond types and atomic valences, but it deliberately disables three-dimensional chirality (`useChirality=False`) and generic feature-based invariants (`useFeatures=False`) unless explicitly invoked.³⁸ In competition environments, while capturing stereochemistry can occasionally yield marginal gains, standard topological representations typically disable it to prevent an uncontrolled explosion in the feature space.

Witness Molecule Evaluation: Ethanol

When applying this circular operation to the witness molecule ethanol ("CCO"), RDKit analyzes the core carbon-carbon and carbon-oxygen connections. Utilizing a radius of 2, the fingerprint aggregator collects information for fragments at radius 0 (the individual carbon and oxygen atoms), radius 1 (the C-C and C-O primary bonds), and radius 2 (the entire C-C-O topological backbone).⁴⁶ The algorithm then folds these sparse identifiers into the 1024-bit vector, resulting in a highly sparse array comprised mostly of zeros, interspersed with ones at specific indices representing the hydroxyl and aliphatic environments. Passing an empty dictionary object to the `bitInfo` parameter during the function call allows a developer to track exactly which atomic indices generated which specific bits—a crucial transparency feature for debugging and for interpreting molecular similarities.³⁸

Implementation Atom Specification: Morgan Fingerprint

Name: `morgan_fingerprint`

Description: Computes a Morgan (ECFP) circular fingerprint bit vector from an RDKit molecule object.

Source:

<https://www.rdkit.org/docs/source/rdkit.Chem.rdMolDescriptors.html#rdkit.Chem.rdMolDescrip>

tors.GetMorganFingerprintAsBitVect

License: BSD-3-Clause (RDKit)

Concept type: analysis

Signature: (mol: MolObject, radius: int, n_bits: int) -> NDArray

Pure function boundary: Takes a read-only MolObject, an integer radius, and a fixed bit length.

Outputs a deterministic, fixed-length 1D NumPy array representing the folded bit vector.

Contains no reliance on external files, database queries, or mutable global state.

Contracts:

- require: mol is not None (valid molecule)
- require: radius >= 0 (standard competitive baseline is 2)
- require: n_bits > 0 (standard competitive baseline is 1024 or 2048)
- ensure: result.shape == (n_bits,)
- ensure: result values are strictly bounded to 0 or 1
- ensure: result dtype is integer or boolean

Witness: ethanol SMILES "CCO" with radius=2, n_bits=1024 generates a sparse vector with verifiable bits set representing the hydroxyl and short aliphatic substructures.

Dependencies: rdkit (BSD-3-Clause), numpy

CDG stages covered: neurips_open_polymer_1st/chemical_descriptor_extraction

Stage 4: MACCS Structural Keys (maccs_keys)

While Morgan fingerprints rely on an algorithmic, mathematically unbounded hashing process to discover substructures organically, MACCS (Molecular ACcess System) keys represent a fundamental alternative philosophy: the dictionary-based fingerprint.¹⁰ Originally developed in the 1970s and 1980s by MDL Information Systems to facilitate rapid database searching, the public MACCS keys constitute a predefined, rigid 166-bit structural dictionary.⁶

The Dictionary Approach vs. Hashing Algorithms

Unlike Morgan fingerprints, which algorithmically catalog every possible topological neighborhood without prejudice, MACCS keys ask 166 highly specific, hardcoded questions about the molecule.¹⁰ Each bit position maps exactly to a predetermined structural fragment or physicochemical property. Examples include boolean inquiries such as: "Are there fewer than 3 oxygens?", "Is there a sulfur-sulfur bond present?", or "Does the molecule contain an aromatic ring?".¹⁰ If the defined substructure or property exists anywhere within the molecular graph, the corresponding bit is set to 1; if absent, it evaluates to 0.

The overriding advantage of MACCS keys lies in their absolute, human-readable interpretability.⁴⁰ Because every bit index strictly corresponds to a known SMARTS (SMILES Arbitrary Target Specification) pattern, a machine learning model attributing high feature importance to a specific MACCS bit provides direct, chemical insight into the mechanism of action. There is zero risk of hash collisions; bit 122 will unequivocally denote the same structural motif regardless of whether the molecule is a simple solvent or a massive biological

macrocycle.⁴⁸

However, the rigidity of MACCS keys severely limits their structural resolution.⁴⁰ The 166-bit dictionary was curated decades ago and is inherently incapable of capturing novel, complex, or exotic substructures that are simply absent from the original specification list. Furthermore, the keys do not inherently capture sequential bond orders or complex stereochemistry effectively.

Despite these apparent limitations, MACCS keys frequently demonstrate remarkable predictive synergy when concatenated alongside Morgan fingerprints in ensemble machine learning models. Because MACCS keys capture high-level, human-curated structural motifs that deeply folded ECFP hashes might inadvertently obscure through chaotic modulo collisions, the two representations provide orthogonal insights into the molecule's nature.¹⁸

RDKit implements the 166 public MACCS keys via the `rdkit.Chem.MACCSkeys.GenMACCSKeys` module.⁴⁸ This function internally executes a series of highly optimized pattern-matching routines, comparing the 166 internal SMARTS patterns against the provided `MolObject`. The RDKit implementation documentation notes a few minor nuances, particularly regarding how aromaticity definitions are handled, which may yield slight discrepancies when compared directly to legacy commercial MDL software implementations.⁴⁸ Furthermore, RDKit's current definitions strictly flag the boolean presence of a match; meaning multiple occurrences of the exact same substructure (e.g., three separate hydroxyl groups) will only trigger a binary 1 at that specific position, without retaining any count information.⁴⁸

Witness Molecule Evaluation: Ethanol

Applying the MACCS key matching algorithm to the witness molecule ethanol ("CCO") requires RDKit to evaluate the small graph against all 166 structural patterns. The algorithm returns a 1 for the specific keys targeting primary alcohols, aliphatic carbon chains, and basic oxygen presence, while leaving the vast majority of the vector's 166 bits as a default 0.⁶ The resulting highly sparse array provides an immediate, coarse-grained snapshot of the molecule's identity.

Implementation Atom Specification: MACCS Keys

Name: `maccs_keys`

Description: Computes the 166-bit structural key fingerprint based on public MACCS SMARTS definitions.

Source: <https://www.rdkit.org/docs/source/rdkit.Chem.MACCSkeys.html>

License: BSD-3-Clause (RDKit)

Concept type: analysis

Signature: `(mol: MolObject) -> NDArray`

Pure function boundary: Takes a read-only `MolObject` and evaluates it against an immutable,

hardcoded list of 166 SMARTS patterns. Outputs a fixed-length vector of 166 (or 167, depending on 0-index padding in the specific RDKit version) integers. No external dependencies, network I/O, or variable global states are invoked.

Contracts:

- require: mol is not None (valid molecule)
- ensure: result.shape == (166,) or (167,)
- ensure: result values are strictly 0 or 1
- ensure: result dtype is integer or boolean

Witness: ethanol SMILES "CCO" generates a highly sparse 166-bit vector with specific, verifiable activations for hydroxyl and short aliphatic motifs.

Dependencies: rdkit (BSD-3-Clause), numpy

CDG stages covered: neurips_open_polymer_1st/chemical_descriptor_extraction

Architectural Integration and Pure-Function Guarantees

Executing these four isolated atoms within a production machine learning pipeline—especially one designed to scale across thousands of polymer candidates—requires rigid adherence to pure-functional programming principles. The central tenet of a pure function mandates that for any given input, the output must consistently remain the same, and the execution must produce zero side effects, such as modifying global variables, mutating the input object, or performing unchecked network I/O operations.⁴

The RDKit C++ backend naturally satisfies the majority of these constraints, but the Python wrapper implementations necessary for the CDG must enforce explicit data type boundaries. For instance, fingerprint generators in RDKit naturally return bespoke, memory-optimized C++ vector objects (such as ExplicitBitVect or UIntSparseIntVect).³⁶ If these specialized objects are allowed to leak out of the functional atom, downstream machine learning libraries will likely crash. These objects must be explicitly unpacked and cast into contiguous NumPy array structures (NDArray) entirely within the atom's local logic boundary. By forcing the cast to an NDArray, the atom ensures that downstream ML frameworks, such as PyTorch, XGBoost, or Scikit-Learn, receive natively compatible dense or sparse tensor formats without requiring secondary bridging functions.⁴⁶

Furthermore, deterministic error propagation must be strictly controlled across the pipeline graph. As previously established, the smiles_to_mol atom returning a None type acts as the pipeline's primary defense mechanism against structural corruption.¹⁴ Consequently, the three downstream analysis atoms (compute_descriptors, morgan_fingerprint, and maccs_keys) all inherently share a critical precondition contract: require: mol is not None. If a None type generated by an invalid SMILES string bypasses the extraction phase, the downstream analysis nodes must immediately halt for that specific molecule, logging the anomaly and dropping the entry or returning a vector of NaNs, rather than throwing a type exception that could crash the

entire parallel execution batch.⁴

Summary of Extraction Methodology

The architectural paradigms detailed throughout this analysis successfully map standard chemical informatics processes onto a formalized, functional directed graph structure. By strictly isolating the data extraction phase—the complex parsing and sanitization of linear SMILES strings—from the computationally intense analytical processing of topological features, overarching computational overhead is minimized. Furthermore, the careful curation of the Top 20 one-dimensional and two-dimensional continuous descriptors prevents arbitrary feature bloat and collinearity while maximizing the transmission of physicochemical signals to the model.

Simultaneously, leveraging the dual representations of Morgan hashed circular fingerprints alongside predefined MACCS dictionary keys provides competitive machine learning algorithms with orthogonal, highly complementary views of chemical space. This dual approach captures both deep, localized connectivity and high-level, human-interpretable structural motifs.¹⁹ Embedding these sophisticated operations within explicit, strongly-typed contract boundaries guarantees the absolute determinism, numerical stability, and high-throughput execution speed required to construct world-class predictive models for polymer screening and advanced chemical discovery.

Works cited

1. 20 Must-Read Papers for a Strong QSAR Foundation in Drug Design [2025] - Neovarsity, accessed May 1, 2026, <https://neovarsity.org/blogs/must-read-qsar-papers-drug-design>
2. MoleculeNet: a benchmark for molecular machine learning, accessed May 1, 2026, <https://rcastoragev2.blob.core.windows.net/a80afbd584722d6dc08c13cd237582bb/PMC5868307.pdf>
3. Fine-tuning GPT-3 for machine learning electronic and functional properties of organic molecules - PMC, accessed May 1, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10762956/>
4. Help for package rdkitpyr - CRAN, accessed May 1, 2026, <https://cran.r-project.org/web/packages/rdkitpyr/refman/rdkitpyr.html>
5. rdkit.Chem.Descriptors module — The RDKit 2025.09.6 documentation, accessed May 1, 2026, <https://www.rdkit.org/docs/source/rdkit.Chem.Descriptors.html>
6. A machine learning method for predicting molecular antimicrobial activity - PMC - NIH, accessed May 1, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11850884/>
7. Basic handling of molecular data using Python - Colab, accessed May 1, 2026, https://colab.research.google.com/github/schwallergroup/ai4chem_course/blob/main/notebooks/01%20-%20Basics/01d_rdkit_basics.ipynb
8. What can Large Language Models do in chemistry? A comprehensive benchmark on eight tasks - arXiv, accessed May 1, 2026, <https://arxiv.org/html/2305.18365v3>

9. Stereoisomers Are Not Machine Learning's Best Friends | Journal of Chemical Information and Modeling - ACS Publications, accessed May 1, 2026,
<https://pubs.acs.org/doi/10.1021/acs.jcim.4c00318>
10. Chemistry resources and tools for compound selection - EMBL-EBI, accessed May 1, 2026,
https://www.ebi.ac.uk/sites/ebi.ac.uk/files/content.ebi.ac.uk/materials/2013/131209DrugDiscovery/4_-_noel_oboyle_-_cheminformatics.pdf
11. Curated Database and Preliminary AutoML QSAR Model for 5-HT1A Receptor - PMC, accessed May 1, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC8536971/>
12. Descriptors Directory Reference - RDKit, accessed May 1, 2026,
https://www.rdkit.org/docs/cppapi/dir_7308569cc8cd2e125f103df034d24117.html
13. rdkit/ReleaseNotes.md at master - GitHub, accessed May 1, 2026,
<https://github.com/rdkit/rdkit/blob/master/ReleaseNotes.md>
14. rdkit.Chem.rdmolfiles module — The RDKit 2025.09.6 documentation, accessed May 1, 2026,
<https://www.rdkit.org/docs/source/rdkit.Chem.rdmolfiles.html#rdkit.Chem.rdmolfiles.MolFromSmiles>
15. Subgraph-based Machine Learning for Graph Generation Masatsugu Yamada, accessed May 1, 2026,
<https://ir.soken.ac.jp/record/2000025/files/A2455%E6%9C%AC%E6%96%87.pdf>
16. Feature-Based Deep Learning Approaches to Facilitate Drug Discovery - University of Wisconsin–Madison, accessed May 1, 2026,
<https://asset.library.wisc.edu/1711.dl/I4V34CHNWTCBX8D/R/file-7d062.pdf>
17. GettingStartedInPython.rst.txt - RDKit, accessed May 1, 2026,
https://www.rdkit.org/docs/_sources/GettingStartedInPython.rst.txt
18. Molecular Descriptors, Structure Generation, and Inverse QSAR/QSPR Based on SELFIES | ACS Omega - ACS Publications, accessed May 1, 2026,
<https://pubs.acs.org/doi/10.1021/acsomega.3c01332>
19. Comparison of Descriptor- and Fingerprint Sets in Machine Learning Models for ADME-Tox Targets - Frontiers, accessed May 1, 2026,
<https://www.frontiersin.org/journals/chemistry/articles/10.3389/fchem.2022.852893/full>
20. Machine Learning on QSAR Data! | 20+ Models XGBoost, LightGBM, CatBoost, TabNet, Neural Nets & More! - YouTube, accessed May 1, 2026,
<https://www.youtube.com/watch?v=BC8r-xb9BpA>
21. Descriptor calculation tutorial – RDKit blog, accessed May 1, 2026,
<https://greglandrum.github.io/rdkit-blog/posts/2022-12-23-descriptor-tutorial.html>
22. Cleaning and selecting the best molecular descriptors for QSAR study (part 1) - YouTube, accessed May 1, 2026,
<https://www.youtube.com/watch?v=hINPS5xHuEc>
23. Full article: Integrated machine learning-driven QSAR modelling and systems biology approach for the identification of potential SARS-CoV-2 3CLpro inhibitors - Taylor & Francis, accessed May 1, 2026,
<https://www.tandfonline.com/doi/full/10.1080/1062936X.2025.2584058>

24. Mold2, Molecular Descriptors from 2D Structures for Chemoinformatics and Toxicoinformatics | Journal of Chemical Information and Modeling - ACS Publications, accessed May 1, 2026, <https://pubs.acs.org/doi/10.1021/ci800038f>
25. QSAR Prediction of BBB Permeability Based on Machine Learning upon PETBD: A Novel Data Set of PET Tracers | Journal of Medicinal Chemistry - ACS Publications, accessed May 1, 2026, <https://pubs.acs.org/doi/10.1021/acs.jmedchem.5c01791>
26. Descriptor-Free Deep Learning QSAR Model for the Fraction Unbound in Human Plasma | Molecular Pharmaceutics - ACS Publications, accessed May 1, 2026, <https://pubs.acs.org/doi/10.1021/acs.molpharmaceut.3c00129>
27. Improving Machine Learning Classification Predictions through SHAP and Features Analysis Interpretation - PMC, accessed May 1, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12606625/>
28. Ranking of the top 20 molecular descriptors in predicting ready... - ResearchGate, accessed May 1, 2026, https://www.researchgate.net/figure/Ranking-of-the-top-20-molecular-descriptors-in-predicting-ready-biodegradability-using_fig5_373379812
29. A Reinforcement Learning-Guided Genetic Algorithm Integrating Medicinal Chemistry-Inspired Molecular Transformations | Journal of Chemical Information and Modeling - ACS Publications, accessed May 1, 2026, <https://pubs.acs.org/doi/10.1021/acs.jcim.6c00397>
30. Extreme Gradient Boosting Combined with Conformal Predictors for Informative Solubility Estimation - MDPI, accessed May 1, 2026, <https://www.mdpi.com/1420-3049/29/1/19>
31. Release 2019.03.1 Greg Landrum - The RDKit Documentation, accessed May 1, 2026, https://rdkit.readthedocs.io/_/downloads/en/release_2019_03/pdf/
32. Supporting Information - ACS Figshare, accessed May 1, 2026, <https://acs.figshare.com/ndownloader/files/48045266>
33. SMILES->RDKit->LGBM->FTW_Group_Cv - Kaggle, accessed May 1, 2026, <https://www.kaggle.com/code/sayedathar11/smiles-rdkit-lgbm-ftw-group-cv>
34. Getting Started with the RDKit in Python — The RDKit 2025.09.6 documentation, accessed May 1, 2026, <https://www.rdkit.org/docs/GettingStartedInPython.html>
35. milanimarcel/Descriptors-from-RDKit - GitHub, accessed May 1, 2026, <https://github.com/milanimarcel/Descriptors-from-RDKit>
36. RDKit Documentation, accessed May 1, 2026, https://www.rdkit.org/RDKit_Docs.2011_12_1.pdf
37. Automatic Prediction of Peak Optical Absorption Wavelengths in Molecules Using Convolutional Neural Networks - PMC, accessed May 1, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10934802/>
38. rdkit.Chem.rdMolDescriptors module — The RDKit 2025.09.6 ..., accessed May 1, 2026, <https://www.rdkit.org/docs/source/rdkit.Chem.rdMolDescriptors.html#rdkit.Chem.rdMolDescriptors.GetMorganFingerprintAsBitVect>
39. Fingerprints in the RDKit, accessed May 1, 2026, https://www.rdkit.org/UGM/2012/Landrum_RDKit_UGM.Fingerprints.Final.pptx.pdf
40. Molecular machine learning past the chemical Kármán Line - Pure, accessed May

- 1, 2026, https://pure.tue.nl/ws/files/369929782/20251111_Tilborg_van_hf.pdf
41. Utility of the Morgan Fingerprint in Structure-Based Virtual Ligand Screening - PMC, accessed May 1, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11163432/>
42. CHEMINFORMATICS - LibreTexts, accessed May 1, 2026, <https://batch.libretexts.org/print/Letter/Finished/chem-143689/Full.pdf>
43. Molecular Representation Matters: Comparative Evaluation of Fingerprints, RDKit Descriptors, and Hashing Effects | The Journal of Physical Chemistry Letters - ACS Publications, accessed May 1, 2026, <https://pubs.acs.org/doi/10.1021/acs.jpcllett.5c03797>
44. Discovery of Inhibitors Targeting p38 γ and PLK1 Using Combined Virtual Screening, Molecular Dynamics Simulations and Biological - Swinburne figshare, accessed May 1, 2026, <https://figshare.swinburne.edu.au/ndownloader/files/47646304>
45. RISC and Dense Fingerprints - ChemRxiv, accessed May 1, 2026, <https://chemrxiv.org/doi/pdf/10.26434/chemrxiv.8218517>
46. Understand Morgan fingerprint bit radius · rdkit rdkit · Discussion ..., accessed May 1, 2026, <https://github.com/rdkit/rdkit/discussions/5826>
47. Compressing Chemistry Reveals Functional Groups | Journal of Chemical Information and Modeling - ACS Publications, accessed May 1, 2026, <https://pubs.acs.org/doi/10.1021/acs.jcim.5c02917>
48. rdkit.Chem.MACCSkeys module — The RDKit 2025.09.6 ..., accessed May 1, 2026, <https://www.rdkit.org/docs/source/rdkit.Chem.MACCSkeys.html>
49. Data Science for drug discovery research -Morgan fingerprints in Python. | by Darko Medin, accessed May 1, 2026, <https://darkomedin-datascience.medium.com/data-science-for-drug-discovery-research-morgan-fingerprints-using-alanine-and-testosterone-92a2c69dd765>